Check for updates

ACM DIGITAL LIBRARY | Association for Computing Machinery | acm open

Latest updates: https://dl.acm.org/doi/10.1145/3746059.3747699

RESEARCH-ARTICLE

Enabling Auto-Correction on Soft Braille Keyboard

DAN ZHANG, Stony Brook University, Stony Brook, NY, United States

YAN MA, Stony Brook University, Stony Brook, NY, United States

GLENN DAUSCH, Stony Brook University, Stony Brook, NY, United States

WILLIAM H SEIPLE

DAVID XIANFENG GU, Stony Brook University, Stony Brook, NY, United States

I V RAMAKRISHNAN, Stony Brook University, Stony Brook, NY, United States

View all

Open Access Support provided by:

Stony Brook University

PDF Download
3746059.3747699.pdf
25 December 2025
Total Citations: 0
Total Downloads: 1078

Published: 28 September 2025

Citation in BibTeX format

UIST '25: The 38th Annual ACM Symposium on User Interface Software and Technology
September 28 - October 1, 2025
Busan, Republic of Korea

Conference Sponsors:
SIGCHI
SIGGRAPH

Enabling Auto-Correction on Soft Braille Keyboard

Dan Zhang
Computer Science Department
Stony Brook University
Stony Brook, New York, USA
zhang64@cs.stonybrook.edu

Yan Ma
Computer Science Department
Stony Brook University
Stony Brook, New York, USA
yanma1@cs.stonybrook.edu

Glenn Dausch
Computer Science
Stony Brook University
Stony Brook, New York, USA
glenn.dausch@stonybrook.edu

William H Seiple
Lighthouse Guild
New York, New York, USA
wseiple@lighthouseguild.org

David Xianfeng Gu
Department of Computer Science
Stony Brook University
Stony Brook, New York, USA
gu@cs.stonybrook.edu

IV Ramakrishnan
Computer Science
Stony Brook University
Stony Brook, New York, USA
ram@cs.stonybrook.edu

Xiaojun Bi
Department of Computer Science
Stony Brook University
Stony Brook, New York, USA
xiaojun@cs.stonybrook.edu

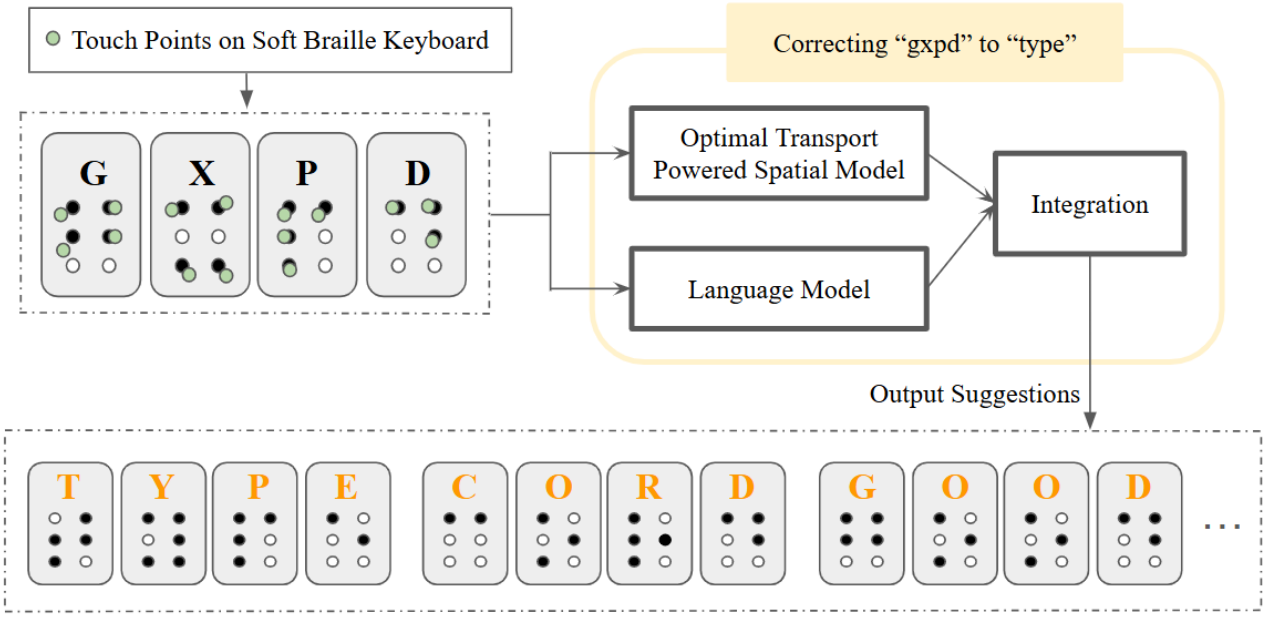


Figure 1: Illustration of word auto-correction on the proposed intelligent Braille keyboard. In this example, the user wants to type word "type", but mistakenly has a dispositional touch for letter 't', an omitted touch for letter 'y', and an extra touch for letter 'e'. The touch input positions are initially mapped to letters: 'g', 'x', 'p', and 'd'. The keyboard then utilizes two modules—spacial model powered by optimal transport and a language model—to compute the likelihood of each candidate word being the intended word. After integrating these probabilities, the keyboard displays the top three candidate words with the highest probabilities at the bottom: "type", "cord", and "good".

Abstract

A soft Braille keyboard is a graphical representation of the Braille writing system on smartphones. It provides an essential text input method for visually impaired individuals, but accuracy and efficiency remain significant challenges. We present an intelligent Braille keyboard with auto-correction ability, which uses optimal transportation theory to estimate the distances between touch input and Braille patterns, and combines it with a language model to estimate the probability of entering words. The proposed system was evaluated through both simulations and user studies. In a touch interaction simulation on an Android phone and an iPhone, our intelligent Braille keyboard demonstrated superior error correction performance compared to the Android Braille keyboard with proofreading suggestions and the iPhone Braille keyboard with spelling suggestions. It reduced the error rate from 55.81% on Android and 57.13% on iPhone to 19.80% under high typing noise. Furthermore, in a user study of 12 participants who are legally blind, the intelligent Braille keyboard reduced word error rate (WER) by 59.5% (42.53% to 17.28%) with a slight drop of 0.74 words per minute (WPM), compared to a conventional Braille keyboard without auto-correction. These findings suggest that our approach has the potential to greatly improve the typing experience for Braille users on touchscreen devices.

CCS Concepts

• **Human-centered computing** → **Accessibility technologies**; **Text input**.

Keywords

Braille Keyboard, Accessibility, Text Entry

ACM Reference Format:

Dan Zhang, Yan Ma, Glenn Dausch, William H Seiple, David Xianfeng Gu, IV Ramakrishnan, and Xiaojun Bi. 2025. Enabling Auto-Correction on Soft Braille Keyboard. In *The 38th Annual ACM Symposium on User Interface Software and Technology (UIST '25), September 28–October 01, 2025, Busan, Republic of Korea.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3746059.3747699

1 Introduction

The smartphone has become inextricably tied to daily life of everyone including people who are blind. However, they face several challenges interacting with smartphones. One major problem is inputting text on smartphones. The common letter-level search-and-confirm method on a QWERTY soft keyboard, the default for blind users on smartphones, typically yields a maximum speed of about 5 Words Per Minute (WPM), an average Word Error Rate (WER) of 25% [13, 15, 43, 45, 62].

A soft Braille keyboard is a graphical representation of the Braille writing system [16] on smartphones, which uses patterns on a 3×2 grids to represent letters and symbols. Figure 2 shows the Braille patterns for 26 letters and how a user types on a Braille keyboard. Although Braille is important, using Braille has been poorly supported on smartphones. The major problem is that current Braille keyboard deterministically maps touch input into a letter, lacking intelligence to correct touch errors. In an evaluation of BrailleTouch, an accessible keyboard for blind users on smartphones, the group of participants with moderate performance had an averaged 40.5% and 33.1% error on the tablet and touchscreen, respectively [26, 52]. The high error rate has prevented blind users from reaping the full benefits afforded by smartphones, as text input is arguably a major task that users often do on their smartphones. In fact it alone accounts for 40% of one's smartphone usage [18], and constitutes the core activity of the top smartphone applications [48] including emailing, messaging, internet browsing, and social media usage.

The modern soft keyboards for sighted users are powered with auto-correction capability to handle input errors. However, such intelligent ability is absent on Braille keyboards for blind users. The statistical decoding technique [27] that empowers a soft keyboard with auto-correction functions, can automatically correct input errors for sighted users. A previous study [25] shows that statistical decoding can reduce text entry errors from 38.4% to 5.7%, a dramatic improvement in input accuracy.

However, the statistical decoding technique [27], which was created for tap typing only on QWERTY keyboards, is not applicable to Braille keyboards. A statistical decoder estimates the probability of entering a letter by calculating the distance between the touch point and a key center, and converting the distance into the probability using a Gaussian model [27]. However, such a method does not work for typing on a Braille keyboard on which letters are represented by dot patterns defined on a 2×3 grid (e.g., A with 1 dot, B with 2 dots, D with 3 dots in Figure 2c). When typing a letter on a Braille keyboard, a user may generate multiple touch points. The existing statistical decoder cannot calculate the probability of entering a letter with multiple touch points. The situation will be more challenging if the number of touch points does not match the number of dots representing a letter. For example, a user may generate three touch points when typing a letter. The statistical decoder will not be able to estimate the probability of entering letter "Q" which is defined by 5 dots (Figure 2c). Therefore, the exiting statistical decoding technique cannot handle Barille input. As a result, there exists a large gap in the text input capabilities of smartphones between sighted and blind users. We expect introducing auto-correction to Braille keyboard would result in even greater error reduction as blind users' touch input might be more error-prone [52].

In this paper, we present an intelligent Braille keyboard that enables auto-correction using optimal transport theory [59], a mathematical framework that computes the minimal cost to transform one distribution into another. We apply this theory to match user touch input with Braille character patterns. Specifically, each Braille character is represented by a fixed pattern of dots arranged on a 2×3 grid. The system treats this pattern as a set of discrete points, each carrying equal weight—analogous to evenly spaced piles of sand. A user's input will generate another set of points, which are also

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

UIST '25, Busan, Republic of Korea

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-2037-6/25/09
https://doi.org/10.1145/3746059.3747699

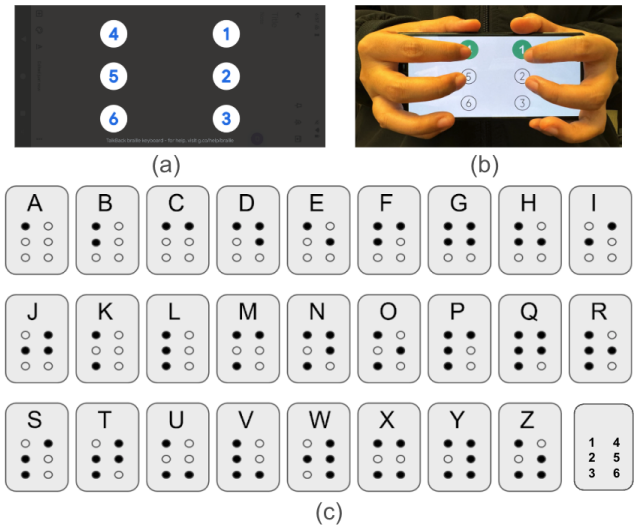


Figure 2: (a) Screenshot of an existing conventional Braille keyboard. (b) Entering 'c' on the intelligent Braille keyboard. (c) Mappings between Braille input and letters, and the numbering of Braille dots. Note that the numbering of Braille dots on the touchscreen mirrors the layout in (c), as the touchscreen is facing away during typing.

considered equally weighted. The system then compares the user's touch points to the dot patterns of all possible Braille characters. Using optimal transport, it calculates the minimum "cost" required to move the mass of the user's touch distribution to align with each candidate Braille pattern. A lower cost indicates a closer spatial match. These transport costs are then transformed into spatial probabilities using a Gaussian function, where a lower cost corresponds to a higher probability. Finally, a Bayesian decoder integrates this spatial probability with a language model to infer and auto-correct the most likely intended letter or word.

We conducted extensive evaluations through both simulations and user studies. The touchpoint interaction simulation revealed that our intelligent keyboard substantially surpasses the performance of the Android Braille keyboard with proofread feature and the iPhone Braille keyboard with an integrated spellchecker. The evaluation was performed with touchpoint inputs subjected to varying levels of noise and diverse error types. In high touch-point noise conditions, where the standard deviation (SD) of Gaussian noise was set to 7.8 mm, our intelligent Braille keyboard demonstrated robust performance in handling input variability, achieving a Top-1 WER of 19.80%, where Top-1 WER refers to the WER with the first suggested word in the Braille keyboard. It is drastically lower than the Android phone's 55.81% and the iPhone's 57.13%, indicating a 64.52% and a 65.34% error reduction respectively. Furthermore, we carried out a user study involving 12 legally blind participants to compare our intelligent keyboard with the conventional Braille keyboard without auto-correction. The results of WER were compelling: our intelligent keyboard reduced WER from 42.53% to 17.28%, marking a substantial 59.5% improvement over the conventional Braille keyboard. In the final block of the user study, participants achieved an average typing speed of 6.81 WPM with the intelligent Braille keyboard, compared to 7.42 WPM with the conventional Braille keyboard—a reduction of 0.61 WPM (8.22%). This slower speed is likely due to the additional time needed to swipe through the suggestion list. Additionally, A higher System Usability Scale (SUS) questionnaire [17] score for our system indicates that users find our intelligent Braille keyboard more user-friendly and satisfactory, which is consistent with the improved WER.

2 Related Work

In this section, we present the existing literature on text entry techniques for visually impaired people, statistical decoding, and on-screen Braille keyboards.

2.1 Text Entry Techniques for Visually Impaired Individuals

Text entry on smartphones is known to be challenging due to the difficulty of precise targeting [52]. Although non-visual text input methods exist, they are inefficient and difficult to use. The common approach is the letter-level search-and-confirm method on a QWERTY soft keyboard, the default text input method on smartphones. This method is extremely slow as it requires repetitive searching and confirmation. A longitudinal study over eight weeks showed that participants improved on average 2.4 WPM (SD=.36) from week one with 1.6 WPM (SD=.23) to 4 WPM (SD=.35) [43]. Other studies have also shown that users can only type around 5 WPM with this method [13, 15, 45, 62], which is 90% slower than their sighted counterparts (36 WPM) [47]. Voice-based text entry (with Siri, for example) is a potentially fast text entry method in circumstances where privacy is not of concern and audible input is socially acceptable. However, correcting errors in spoken text is challenging for blind users [8, 32]. The current error correction method requires users to first precisely position the cursor at the error location, and then type on the keyboard. Due to the absence of visual feedback, such operations are difficult to do for blind users. Previous research [32] has shown that the lack of efficient error location and correction methods is the main impediment for the adoption of voicing-based text input, as blind users often spend more than 80% of their time in correcting errors in spoken text [8]. The third option is a Braille soft keyboard (e.g., [35]), which is a graphical representation of a Braille layout on a touchscreen. A user enters a letter by pressing the keys on a Braille layout according to its corresponding Braille dot pattern. However, touch input is often inaccurate and error-prone because of the lack of visual feedback and the flat contact area of finger touch [11, 31]. Typing on such a layout introduces errors, but extant Braille soft keyboards (e.g., [35]) do not provide automatic error correction.

2.2 Statistical Decoding

Improving virtual keyboard decoding has been intensively investigated by HCI researchers in order to handle noises in touch input. A common approach is to first convert a sequence of touch points into a sequence of keys and then, search for the most likely match between the key sequence and a word. In modern keyboards, a significant amount of research has adopted a statistical approach: the keyboard decoding algorithm handles touch points at each key

as signals that follow a certain probability distribution over a spatial region. The Bayes rule is applied to combine spatial probabilities inferred from touch points with prior probabilities obtained from language models to calculate the probability for word candidates. Goodman et al.'s [27] work was the first to integrate a language model with a pen/touch model. The dual Gaussian model [12, 37] provided a more effective approach to model finger touch locations for text entry. Vertanen et al. [57] developed a touchscreen keyboard decoder that supports a sentence-based text entry approach. Zhu et al.'s work investigated the feasibility of an invisible keyboard being a design option, where they adapted the spatial model in the statistical decoder to accommodate an invisible virtual keyboard [63]. Researchers also proposed other statistical approaches for keyboard decoding other than leveraging the language model. Kristensson and Zhai's research [36, 61] posed a pattern-matching approach that treated touch points as high-resolution geometric patterns, which were matched to patterns produced by key center positions of legitimate words in a lexicon. A key-target resizing algorithm was proposed by Gunawardana et al. [29], which dynamically resized the underlying target areas on their probabilities. However, these innovations have not been applied to blind users, creating a gap in text input experiences between sighted and visually impaired users.

2.3 On-screen Braille Keyboard Techniques

Existing on-screen Braille Keyboard techniques focus on the exploration of layout design and interaction methods. BrailleType [45] and BrailleTouch [26] split the screen into six sections for the Braille dots, while the latter enables multi-touch techniques that users type all the dots for a letter simultaneously. TypeInBraille [39] is a typing technique with three gestures that each gesture decides one row of the Braille dots. BrailleKey [53] improves on BrailleTouch by using four large corner buttons for text entry. Azenkot et. al [7, 9] proposed a touchscreen input detection method to detect the input fingers and track the hand drift while typing Braille. EdgeBraille [40, 41] enables six-point Braille entry by swiping a finger along the screen edges in any sequence. SingleTapBraille [1] allows Braille text entry by tapping anywhere on the screen multiple times, following Braille patterns. In BrailleEnter [2], users input a letter by tapping or pressing on a touchscreen six times, regardless of the location on the screen. Braille keyboards were integrated into iOS devices starting with iOS 8, released in September 2014 [34]. Android added on-screen Braille keyboards via apps like Google BrailleBack and MBraille since Android 5.0 in November 2014 [28]. In April 2020, Google natively integrated this feature as the TalkBack Braille Keyboard [55]. These keyboards help users type faster and more accurately without external keyboards.

Limited research is delicated to error corrections for on-screen Braille keyboard. Smartphones feature built-in spell checkers that suggest corrections for misspelled words. iOS extended spellchecker to Braille keyboards, allowing users to swipe through spelling suggestions. The Android Braille keyboard introduced typo correction in proofread mode since TalkBack 14.1. B# [42] stands out as a chord-based correction system that reduces Braille keyboard error rates by word similarity analysis. Our method takes a different approach by leveraging an intelligent Braille keyboard with auto-correction that dynamically interprets user input patterns to provide more accurate corrections. This approach is based on Optimal Transport [59] theory and offers a context-aware error correction mechanism for on-screen Braille keyboard.

2.4 Optimal Transport and Its Applications

Optimal Transport (OT) is a mathematical framework for comparing probability distributions by computing the minimal cost of transforming one distribution into another. Often described through the earth mover's distance metaphor, OT models the effort needed to redistribute one set of points (e.g., sand piles) into another (e.g., holes) while minimizing a transport cost [59].

OT has gained prominence across diverse fields such as computer vision, machine learning, and Human-Computer Interaction (HCI), primarily due to its robust capability to handle structural mismatches and spatial variability between distributions. Its applications are widespread, including image retrieval, domain adaptation [20], generative modeling [4], and spatial input modeling [33]. Within HCI, OT is particularly valuable for comparing complex, real-world inputs like multi-touch patterns or free-form gestures, which are often characterized by noise, incompleteness, or variations in the number of contact points—challenges directly analogous to those encountered in soft Braille keyboard interaction.

Our Braille decoding task fits naturally within this OT framework, where user touchpoints are modeled as a discrete distribution and compared to fixed Braille dot patterns. We use the squared Wasserstein-2 distance, which calculates how much "effort" it takes to match the user's touchpoints to a Braille letter pattern. This method can handle common issues in Braille typing, such as when the number of touchpoints doesn't match the number of dots in a letter, or when touches are far from the ideal positions. In contrast, traditional models based on Gaussian distributions or simple distances often fail in these cases [24, 49].

3 Intelligent Braille Keyboard

A soft Braille keyboard (e.g., Android's built-in Braille keyboard [35], Figure 2a) is a graphical representation of a Braille layout on touchscreen, where users enter a letter by pressing the keys on the Braille layout according to its corresponding Braille dot pattern (Figure 2b). We proposed an intelligent Braille keyboard with auto-correction that leverages both touch-point locations and contextual information from the typed text.

3.1 Decoding Principle

Decoding is the process of converting input signals into intended words. The decoding principle for Braille input is similar to the statistical decoders [27, 63] used on a virtual QWERTY keyboard. It consists of two components: spatial and language models. The spatial model infers the probability of entering a word based on the spatial signals such as the locations of touch points, and the language model infers the probability of a word based on the language context. The probabilities from both components are combined via the Bayes' theorem to form the posterior probability of entering a word. The word with the highest posterior probability becomes the top suggestion.

The major challenge of decoding Braille input lies in the spatial model. The commonly used Gaussian model for a QWERTY keyboard does not work for Braille input. On a QWERTY keyboard, each letter is entered with one touch point only. Given a touch point on a QWERTY keyboard, the spatial model calculates the probability of entering a letter by assuming the touch point distribution of entering a letter is a Gaussian distribution. However, such a method does not work for Braille input, because the number of observed touch points might not match the number of dots of a Braille letter. As the example shown in Figure 3, the user inputs 4 touch points on the touchscreen, while the letter c is represented by only 2 dots. The commonly used Gaussian model cannot estimate the probability of entering c with 4 touch points.

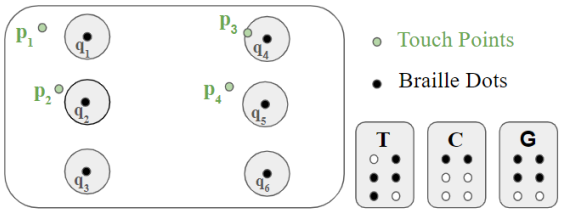


Figure 3: Illustration of Braille input and the Braille dot patterns of three candidate characters, 't', 'c' and 'g', for the example in Section 3.2 and 3.3.

We adopted the optimal mass transport theory [59] to address this challenge. We first represent Braille letters and user input as discrete mass distributions. We then calculate the distance between the user's input and a Braille letter pattern by formulating it as an optimal mass transportation problem: we are seeking the minimum transportation cost of transforming user's input which is represented as a mass distribution to a Braille letter which is represented as another mass distribution. This process can be explained with a metaphor of moving soil. The mass distribution representing the user input is the initial distribution of 1 unit of soil. We are seeking the minimum transportation cost of redistributing this amount of soil to match another mass distribution which represents a Braille letter.

Our method can handle the scenario in which the number of input touch points does not match the number of the dots of a Braille letter: it means the two discrete mass distributions have different number of dots with masses. The optimal transport theory can well handle such a case as explained through an example in Section 3.3. Next, we provide technical details and an example of this process.

3.2 Representing User Input and Braille Patterns as Distributions

First, we represent the user input as a discrete mass distribution. We basically assume that the mass is evenly distributed among the touch points: the mass at each touch point is $\frac{1}{m}$. More formally, assuming that the user generates m touch points for entering a letter, we use $\mathbf{a} = [a_1, a_2, \ldots, a_m]$ and $\mathbf{p} = [p_1, p_2, \ldots, p_m]$ to represent this distribution. Each a_i represents the mass with the value of $\frac{1}{m}$, and each p_i represents the location of the mass, which is the location of the corresponding touch point.

Second, we also represent a Braille character (denoted by c) as a discrete mass distribution. Each letter is mapped to a group of n black dots located on a 2 by 3 grid according to Figure 2c, the mass for each black dot is $\frac{1}{n}$, and the location of each black dot is the center of the corresponding grid. More formally, we use $\mathbf{b}(c) = [b_1, b_2, \ldots, b_6]$ to represent the mass at each Braille dot, and $\mathbf{q} = [q_1, q_2, \ldots, q_6]$ to represent the positions of the six Braille dots on the screen. $\mathbf{q}$ is a constant as the Braille dots have fixed positions. If the Braille letter c is represented by n Braille dots, the mass of the corresponding b_i will be $\frac{1}{n}$, and the mass of other elements in $\mathbf{b}$ will be 0.

We provide an example of calculating squared Wasserstein-2 distance (W_2^2) between user input (green dots in Figure 3) and letters 't', 'c' and 'g'.

We first use $\mathbf{a}$ and $\mathbf{p}$ to define the mass distribution representing user input which consists of 4 touch points (four green dots in Figure 3):

$$\mathbf{a} = [a_1, a_2, a_3, a_4] = [\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}] \tag{1}$$

$$\mathbf{p} = [p_1, p_2, p_3, p_4] \tag{2}$$

$$= [(350.1, 254.9), (217.1, 277.3), (348.6, 576.0), (228.2, 559.2)] \tag{3}$$

where a_i denotes the mass at touch point location p_i. Equations 1, and 3 suggest that mass is evenly distributed among the 4 touch points.

Given that the positions of 6 Braille dots are represented by $\mathbf{q}$:

$$\mathbf{q} = [q_1, q_2, q_3, q_4, q_5, q_6] \tag{4}$$

$$= [(342, 300), (206, 300), (70, 300), (342, 600), (206, 600), (70, 600)] \tag{5}$$

we have $\mathbf{b}$('t'), $\mathbf{b}$('c'), and $\mathbf{b}$('g') as follows:

$$\mathbf{b}(\text{'t'}) = [0, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}, 0] \tag{6}$$

$$\mathbf{b}(\text{'c'}) = [\frac{1}{2}, 0, 0, \frac{1}{2}, 0, 0] \tag{7}$$

$$\mathbf{b}(\text{'g'}) = [\frac{1}{4}, \frac{1}{4}, 0, \frac{1}{4}, \frac{1}{4}, 0] \tag{8}$$

Equation 6 suggests that the mass is evenly distributed at Braille dots q_2, q_3, q_4, and q_5 for 't', Equation 7 suggests that the mass is evenly distributed at Braille dots q_1, and q_4 for 'c', and Equation 8 suggests that the mass is evenly distributed at Braille dots q_1, q_2, q_4, and q_5 for 'g'. They all match the Braille patterns of 't', 'c', and 'g' (Figure 3).

3.3 Squared Wasserstein-2 Distance Between Touch Input and Braille Pattern

The squared Wasserstein-2 distance W_2^2 is widely used in optimal transport problems [50], as squaring often leads to fast convergence [33, 51]. This metric uses the squared Euclidean distance as the cost function—intuitively modeling the effort required to move "mass," like sand, from one location to another. This choice

captures spatial structure naturally, penalizes longer transport distances more heavily, and avoids non-differentiability at zero (unlike the unsquared version), which improves optimization stability and convergence. The process of calculating W_2^2 between user input and a Braille letter consists of the following steps.

First, we generate the cost matrix $M \in \mathbb{R}^{m\times 6}$ by calculating the pairwise squared Euclidean distances between the user's touch points and the standard positions of the six Braille dot blocks. This cost matrix captures the geometric relationship between the user's input and the Braille keyboard layout. M_{ij} reflects the cost of moving a unit mass from p_i to dot q_j, which is defined as the squared Euclidean distance between p_i and q_j:

$$M_{ij} = \|p_i - q_j\|^2. \tag{9}$$

Second, we calculate the squared Wasserstein-2 distance, denoted by $W_2^2(\mathbf{p}, c)$, between the mass distribution representing user input, and the mass distribution representing the Braille letter c, given the cost matrix M defined in Equation 9. It can be formulated as an optimal transportation problem: we are seeking the minimum transportation cost of transforming user's input which is represented as a mass distribution to a Braille letter which is represented as another mass distribution.

$$W_2^2(\mathbf{p}, c) = \min_{\gamma\in T} \sum_{i=1}^{m}\sum_{j=1}^{6} M_{ij}\gamma_{ij}, \tag{10}$$

where T is the set of all possible transportation plans. Equation 10 is a linear problem [59], and there exist a number of solvers [3, 14, 21]. We solve it using the Python Optimal Transport (POT) package with the method of network simplex [24, 56], obtaining $W_2^2(\mathbf{p}, c)$, as well as the optimal transportation plan γ^*. This plan γ^* is represented by a $m \times 6$ matrix. Each element γ^*_{ij} in γ^* suggests the amount of mass transported from touch point p_i to Braille dot q_j.

Using the same example shown in Figure 3, we compute the cost matrix M according to Equation 9 with $\mathbf{p}$ defined in Equation 3 and $\mathbf{q}$ defined in Equation 5:

$$M = \begin{bmatrix} 2103 & 22801 & 80491 & 119189 & 139887 & 197577 \\ 16103 & 638 & 22165 & 119703 & 104238 & 125765 \\ 76219 & 96503 & 153778 & 619 & 20903 & 78178 \\ 80157 & 67697 & 92229 & 14614 & 2153 & 26686 \end{bmatrix} \tag{11}$$

Third, we use the POT package [24] with $\mathbf{a}$, $\mathbf{b}$ and M to obtain the optimal transportation plan γ^* and Squared Wasserstein-2 distance W_2^2 for 't', 'c', and 'g':

$$\gamma^*(\text{'t'}) = \begin{bmatrix} 0 & \frac{1}{4} & 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{4} & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{4} & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{4} & 0 \end{bmatrix} \tag{12}$$

$$W_2^2(\mathbf{p}, \text{'t'}) = 11934 \tag{13}$$

$$\gamma^*(\text{'c'}) = \begin{bmatrix} \frac{1}{4} & 0 & 0 & 0 & 0 & 0 \\ \frac{1}{4} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{4} & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{4} & 0 & 0 \end{bmatrix} \tag{14}$$

$$W_2^2(\mathbf{p}, \text{'c'}) = 8360 \tag{15}$$

$$\gamma^*(\text{'g'}) = \begin{bmatrix} \frac{1}{4} & 0 & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{4} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{4} & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{4} & 0 \end{bmatrix} \tag{16}$$

$$W_2^2(\mathbf{p}, \text{'g'}) = 1378 \tag{17}$$

Each element γ^*_{ij} in γ^* suggests the amount of mass transported from touch point p_i to Braille dot q_j. Regarding 't', Equation 12 suggests that the optimal transport plan is to move $\frac{1}{4}$ of total mass from p_1 to q_2, from p_2 to q_3, from p_3 to q_3, and from p_4 to q_5. Under such a plan, the cost is 11934 (Equation 13). Regarding 'c', the optimal transport plan (Equation 14) is to move $\frac{1}{4}$ mass from p_1 and p_2 to q_1, and from p_3 and p_4 to q_4. Under this plan, the transportation cost is 8360 (Equation 15). Regarding 'g', the optimal transport plan (Equation 16) is to move $\frac{1}{4}$ mass from p_1 to q_1, from p_2 to q_2, from p_3 to q_4, and from p_4 to q_5. The transport cost under this plan is 1378 (Equation 17). Among letters 't', 'c', and 'g', the results suggest that the input in Figure 3 has the shortest distance to 'g', followed by 'c' and 't'.

The computational complexity of decoding a Braille input sequence $\mathcal{P}$ with k Braille input ($\mathcal{P} = (\mathbf{p_1}, \mathbf{p_2}, \ldots, \mathbf{p_k})$) into a word w is $O(k)$. This complexity arises from the need to calculate the squared Wasserstein-2 distance, which has a complexity of $O(n^3)$ with the network simplex method in POT package [56], where n is the number of points in the discrete distributions. As the total number of points in distributions representing user input and a Barille letter is at most 14: six fixed points for the distribution representing a Braille letter and at most eight points for the distribution representing input (4 fingers for each hand while holding the phone as shown in Figure 2b), calculating $W_2^2(\mathbf{p}, c)$ is a small-scale problem and its time complexity can be approximated as a constant $O(1)$. Since decoding a Braille input sequence $\mathcal{P} = (\mathbf{p_1}, \mathbf{p_2}, \ldots, \mathbf{p_k})$ requires calculating $W_2^2(\mathbf{p}, c)$ for each of k Braille inputs $\mathbf{p_i}$, the overall computational complexity is $O(k)$.

3.4 Obtaining Spatial Score: Probability of Entering a Word Based on Touch Input Only

After obtaining the squared Wasserstein-2 distance $W_2^2(\mathbf{p}, c)$ between user input ($\mathbf{p}$) and a Braille letter (c), we convert it into the probability using a Gaussian probability density function:

$$P(c|\mathbf{p}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{W_2^2(\mathbf{p}, c)}{2\sigma^2}\right), \tag{18}$$

where σ is a parameter that controls the spread of the Gaussian function. This probability reflects the probability of entering letter c given the touch input $\mathbf{p}$.

We extend Equation 18 to estimate the probability of entering a word, given a sequence of input points. Assuming the keyboard receives a sequence of Braille input $\mathcal{P} = (\mathbf{p_1}, \mathbf{p_2}, \ldots, \mathbf{p_k})$ where $\mathbf{p_i}$ is a Braille input which might have multiple touch points, the probability of entering the word w with k letters ($w = (c_1, c_2, \ldots, c_k)$) is calculated as follows:

$$P(w|\mathcal{P}) = \prod_{i=1}^{k} P(c_i|\mathbf{p_i}), \tag{19}$$

where $P(c_i|\mathbf{p_i})$ is calculated according to Equation 18. Here we assume that $\mathbf{p_i}$ in $\mathcal{P}$ corresponds to entering the letter c_i of w, and each Braille input $\mathbf{p_i}$ is independent.

Lastly, the spatial score for the word w, which represents the probability of entering w based on touch input $\mathcal{P}$, is obtained by normalizing $P(w|\mathcal{P})$ over the word candidate set W:

$$\mathrm{S}(w) = \frac{P(w|\mathcal{P})}{\sum_{w_j \in W} P(w_j|\mathcal{P})}. \tag{20}$$

3.5 Combining Spatial Score with Language Model Score

Similar to the commonly used statistical decoder [27], we combine the spatial score $\mathrm{S}(w)$ with the probability estimated from a language model, referred to as language model score $L(w)$, to form the final score of entering word w, represented as $Score(w)$:

$$Score(w) = \frac{L(w)S(w)}{\sum_{i \in W} L(i)S(i)}, \tag{21}$$

where W is a lexicon that contains i words. We used a bigram language model (size: 7 million bigrams) to obtain $L(w)$, trained on the Corpus of Contemporary American English (COCA) [22] (2012 to 2017). Our decoding algorithm computes $Score(w)$ for each word and outputs the top N words with the highest scores. We chose $N = 6$ in our implementation.

4 Comparison with Android Typo Correction and iPhone Spellcheckers

We conducted a comparative analysis of the proposed Braille input decoder against the proofread feature [30] in Android's TalkBack Braille keyboard (since TalkBack 14.1) and the spellchecker [54] in iPhone's VoiceOver Braille keyboard via simulating touch interactions on the touchscreens of an Android phone and an iPhone.

4.1 Simulation Setup

To evaluate the performance of the Braille typing auto-correction, we conducted a controlled simulation that closely replicated real-world typing scenarios on both platforms. Instead of relying on human participants, we used automation tools, AutoTouch [6] and Appium [19], to ensure consistency and reproducibility across trials. This approach allowed us to compare our method against Android's proofread feature and iPhone's spelling suggestion feature in a standardized manner. By recording both raw input and word suggestions, we ensured a comprehensive evaluation of each platform's error correction capabilities. For the simulation, we used a diverse phrase set designed for text entry research [58]. This memorable test set comprises 200 sentences, totaling 1074 words. In each sentence, each word was tested once as a trial, with contextual sentence segment prior to the target word entry included. It is important to note that the inclusion of designed noise or errors did not preclude the possibility of correct word input. Therefore, when calculating the WER using the Top-1 suggestion, a trial was only considered an error if neither the typed word nor the first suggestion from the Braille keyboards matched the target word.

For simulating typing interactions on an iPhone, we utilized AutoTouch [6], a scripting tool for automating touch gestures, including multi-finger touch and multi-finger swipe events at the specified coordinates on the screen. The simulations for iPhone spellchecker were performed on an iPhone 6s, operated with iOS version 15.7.3 (released in January, 2023). We choose iPhone 6s because AutoTouch [6], the tool essential for our simulation, is compatible only with earlier versions of iOS (iOS 15 or lower) [5]. This iPhone 6s has a native resolution of 750×1334 pixels. The interaction flow involved:

(1) **Typing Simulation**: Within the Notes application, AutoTouch executed pre-defined scripts to simulate Braille keyboard inputs at the specified coordinates. In each trial of a target word, the script initially and accurately entered the preceding sentence context prior to the target word, followed by the input of the target word with intentionally introduced noise or errors.
(2) **Spellchecker Interaction**: After each word was typed, a swipe-up gesture was simulated to cycle through spelling suggestions, which is the standard interaction on the iPhone Braille keyboard.
(3) **Data Recording**: The misspelled version of the target word and the top three suggestions from the spellchecker were recorded. All results were stored as text entries within the Notes app.

Regarding the simulation on Android, we used Appium [19], a widely used automation framework for UI testing. Appium enables direct interaction with the Android UI, allowing us to programmatically type text, trigger gestures, and extract on-screen content. The simulations were performed on a Google Pixel 6 running Android 14 with TalkBack version 15.1 enabled. The device has a display resolution of 1080×2400 pixels. The typing process followed these steps:

(1) **Typing Simulation**: Appium executed touch inputs on the Android Braille keyboard, entering words into the Notein app, a third-party note-taking application. For each trail of a target word, the script first entered the preceding sentence context accurately, followed by the target word with designed noise or error to replicate realistic typing errors.
(2) **Proofread Mode Activation**: After typing each word, a three-finger swipe left gesture was simulated to enter proofread mode, where proofreading suggestions are provided.
(3) **Suggestion Capture**: A swipe-right gesture was used to cycle through proofreading suggestions. The misspelled target word and the selected suggestion were recorded by reading the onscreen content.
(4) **Return to Typing Mode**: Finally, a three-finger swipe right gesture was simulated to exit proofread mode and resume typing. All results were stored as text entries in a log file.

4.2 Simulation I: Statistical Noise

In this simulation, we aimed to model realistic Braille typing errors by introducing Gaussian noise to touch point coordinates. Each input letter was first mapped to its corresponding Braille representation using the standard Braille alphabet table. Subsequently, a list of touch points, based on the onscreen Braille dot coordinates,

was generated for each letter. To simulate the natural variability in human touch accuracy, we applied Gaussian noise to these touch points [12, 37].

Crucially, we defined the SD of the Gaussian noise in terms of physical displacement rather than pixels to ensure consistency across devices with different screen densities. Specifically, we set the SD to 7.8 mm for the high-noise condition and 3.9 mm for the low-noise condition. These physical values were then converted to pixels based on each device's pixel density, measured in pixels per inch (PPI): for the iPhone 6s (326 PPI), this corresponded to SDs of approximately 100 pixels (7.8 mm) and 50 pixels (3.9 mm); for the Pixel 6 (411 PPI), the SDs were approximately 126 pixels and 63 pixels, respectively. As the intelligent Braille keyboard simulation utilized iPhone Braille dot coordinates, it adopted the iPhone's SD setting of 50 and 100 pixels.

Table 1: Simulation I results: WER (%) with Gaussian noise on Android Braille keyboard, iPhone Braille keyboard, and our intelligent Braille keyboard with auto-correction

Braille	**SD = 7.8 mm**		**SD = 3.9 mm**	
Keyboards	Top-1	Top-3	Top-1	Top-3
Android	55.81	52.74	7.49	6.82
iPhone	57.13	52.82	8.63	7.96
Auto-correction	**19.80**	**12.29**	**1.30**	**0.00**

The results of Simulation I, presented in Table 1, demonstrate that our intelligent Braille keyboard outperforms the Android Braille keyboard and the iPhone Braille keyboard under both high and low Gaussian noise conditions. Under high noise conditions (SD = 7.8 mm), the Android Braille keyboard exhibits a Top-1 error rate of 55.81% and a Top-3 error rate of 52.74%, whereas the iPhone Braille keyboard displays a Top-1 error rate of 57.13% and a Top-3 error rate of 52.82%, suggesting relatively elevated error rates in such scenarios. Conversely, our Intelligent Braille Keyboard presents a markedly lower Top-1 error rate of 19.80% and a Top-3 error rate of 12.29%, indicating its robustness in accommodating greater deviations in touch-point inputs. Similarly, under low noise conditions (SD = 3.9 mm), the Intelligent Braille Keyboard maintains its superior performance with almost negligible error rates (Top-1: 1.30%, Top-3: 0.00%), outperforming both the Android and iPhone Braille keyboards, which exhibit lower but still appreciable error rates.

These results highlight the effectiveness of the proposed Braille typing auto-correction algorithm. The consistently lower error rates of the decoder suggest that it provides a more reliable correction mechanism compared to the existing Android Braille keyboard's proofread feature and iPhone Braille keyboard's spellchecker.

4.3 Simulation II: Adding a Specific Error Type

Not every touch point with noise lead to an error in typing. To comprehensively assess the algorithm's performance, we considered three types of typing errors that may occur while typing on a Braille keyboard.

(1) **Disposition:** These occur when users attempt to press the correct Braille dots but accidentally mistouch one or more dots due to the imprecise nature of finger touch.
(2) **Omission:** Instances where the user fails to touch one or more of the required Braille dots, resulting in incomplete input.
(3) **Extra:** Situations where additional, unintended dots are touched, leading to incorrect input.

For each type of error, we generated the corresponding error patterns and incorporated them into our simulation. For each word, we meticulously pick a letter in the word to add one type of error. We consider the single dot errors in this simulation as it is the the dominant error occurred in typing [42]. For a specific letter, we first obtain its exact coordination list on the screen, which are in the center of the corresponding Braille dots. Then we (1) randomly pick a coordination to replace it with another Braille-dot coordination which is not in the coordination list, (2) delete a randomly picked coordination from the coordination list, (3) add an extra coordination of the Braille dot to the coordination list which is not in the list. By running simulations with these varied error types, we aim to evaluate how well the auto-correction algorithm handles different types of input inaccuracies and to compare its performance with the iPhone spellchecker effectively. The results are summarized in Table 2.

Table 2: Simulation II results: WER (%) for handling different typing errors on Android Braille keyboard, iPhone Braille keyboard, and our intelligent Braille keyboard

Braille	**Disposition**	**Omitted**	**Extra**
Keyboards	Top-1/ Top-3	Top-1/ Top-3	Top-1/ Top-3
Android	64.79/ 60.02	64.41/ 60.50	59.06/ 54.58
iPhone	55.29/ 46.46	48.58/ 37.85	47.36/ 34.39
Auto-correction	**19.54/ 10.49**	**11.49/ 6.90**	**14.02/ 8.04**

For "One Disposition Dot" errors, the Intelligent Braille Keyboard substantially outperformed both the Android and iPhone Braille keyboards, achieving a Top-1 WER of 19.54% compared to 64.79% and 55.29%, respectively. Similarly, for "One Omitted Dot" errors, it showed a Top-1 WER of 11.49%, whereas the Android and iPhone keyboards had 64.41% and 48.58%. For "One Extra Dot" errors, the Intelligent Braille Keyboard maintained superior performance with a Top-1 WER of 14.02%, compared to 59.06% for Android and 47.36% for iPhone.

These results highlight the decoder's superior robustness to various error types compared to the iPhone's spellchecker, suggesting a more reliable solution for Braille typing on touchscreens.

Figure 4 shows the WER on Top-3 suggestions across different word lengths. The analysis is based on the entire simulated dataset, which includes all error types listed in Table 2. As illustrated, the Intelligent Braille keyboard is more accurate than the Android Braille keyboard with proofreading suggestions and the iPhone Braille keyboard with spelling suggestions. It confirms the findings shown in Table 2.

4.4 Simulation III: Multi-letter Errors

According to our observation and the results from simulation I and simulation II, the Android Braille's proofreading feature and

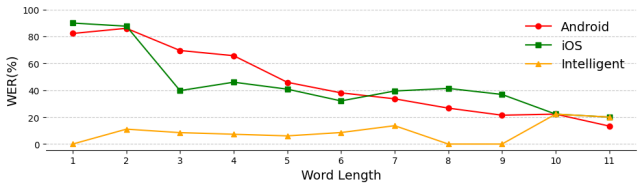


Figure 4: WER on Top-3 Suggestions Across Different Word Lengths

iPhone Braille's spelling suggestion feature can rarely correct multi-letter errors in a word. In the case of multiple letters having errors, we only run a simulation on our Braille auto-correction algorithm.

Table 3: Simulation III results: WER (%) results with two letters mistyped in a word on our Intelligent Braille keyboard

	Disposition	Omitted	Extra
Top-1	34.79	19.08	17.93
Top-3	20.96	8.59	5.06

The table shows the WER with two mistyped letters in each word for different error types under Top-1 and Top-3 criteria. "One Disposition Dot" errors have the highest WER (Top-1: 34.79%, Top-3: 20.96%), indicating difficulty in correcting dispositional mistakes. In contrast, "One Extra Dot" errors have the lowest WER (Top-1: 17.93%, Top-3: 5.06%), suggesting the decoder better handles extra dot errors. Overall, the decoder struggles more with disposition errors than with extra or missing dots.

5 Evaluation of the Intelligent Braille Keyboard

The goal of the evaluation was to understand whether our intelligent Braille decoder improved the typing performance, compared with a Barille keyboard without auto-correction. We conducted an IRB-approved user study to evaluate the performance of our intelligent Braille keyboard with a conventional Braille keyboard without the auto-correction feature. More specifically, our baseline was a conventional TalkBack Braille keyboard without using its spellcheck mode.

5.1 Participants and Apparatus

We recruited 12 participants (6 women, 6 men) between 30 and 65 years of age (average 49.3). All the participants were legally blind and Braille-knowledgeable. Participants were asked to report how often they type Braille, with response options including "Daily", "Weekly", "Monthly", "Rarely", and "Never". Among the participants, five participants reported typing Braille daily, followed by four participants who reported weekly use. Monthly usage was noted by three participants. No participants selected "Rarely" or "Never". They were recruited from a non-profit vision and healthcare organization. The study was IRB-approved and all the participants participated with informed consent. All participants own a touchscreen smartphone. However, none had prior experience with Braille keyboard on smartphones, either Android or iPhone. Their prior experience was with physical input devices such as Braille notetakers (e.g., BrailleNote, BrailleSense), manual tools (e.g., Perkins Brailler, slate and stylus).

We conducted the study on a Google Pixel 6 smartphone running Android 14 with a 2400×1080 pixels display in this study. We created a web-based Braille keyboard as the prototype for our proposed intelligent Braille keyboard. The participants accessed the system using the Chrome browser on the provided smartphone. Participants were instructed to type in screen-away mode, holding the device facing away so their fingers curl backwards to reach the screen. We chose the screen-away mode, a standard configuration for on-screen Braille keyboards, as it aligns with the conventional method for Braille typing on smartphones. Figure 5 shows that a participant is inputting on our proposed intelligent keyboard prototype. Users simultaneously press the desired number of points to enter the letter.

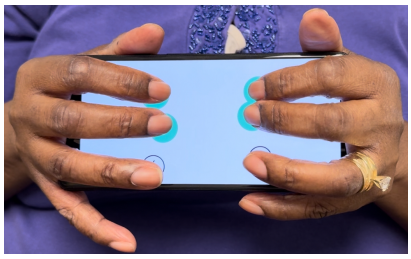


Figure 5: A user is typing letter "g" in screen-away mode on the intelligent Braille keyboard

5.2 Prototype of the Intelligent Braille Keyboard

Our prototype Braille keyboard with Braille layout interface implemented is shown in Figure 2b. The web-based keyboard is presented in full-screen mode to mimic the appearance of the system's default Braille keyboard, eliminating the browser address bar and settings as shown in Figure 5. The Braille dots are arranged in a grid of two parallel vertical columns, each containing three dots. From the user's perspective, the left side contains Braille dots one, two, and three, while the right side contains dots four, five, and six. Users can type by pressing three fingers from each hand on the corresponding dots. Only grade-1 Braille is supported in this current implementation.

Before typing, a calibration is required where the user places all six typing fingers on the screen simultaneously. The system calibrates these positions to match the Braille dots, and the user can then release their fingers. A vibration confirms the calibration is complete. When a combination of dots is typed on the Braille keyboard, the corresponding character is read out. To type a space, users swipe right with one finger. The core component of the web-based Braille keyboard is a statistical decoder that can auto-correct Braille input. Swiping up or down cycles through the suggested words provided by our algorithm, which are read aloud. The suggestion function offers a list of up to six words. After hearing the intended word, the user swipes right to select it, which also adds a space automatically. If the intended word is not found, the user can

continue to the original typed word at the end of the suggestion list.

Similar to the study about Braille input correction [42], our user study adopted a typing mode in which users typed ahead without manually correcting errors. The rationale was to isolate and directly measure the impact of the auto-correction algorithm. This mode collected uncorrected input errors, allowing us to evaluate the correction ability of the algorithm. Allowing backspace could reduce or even eliminate errors, which would limit the ability to meaningfully test performance. The system recorded the following data for analysis: timestamp, multi-touchpoint positions, action type (add a letter, add a space, or add a suggested word), and the cumulative input text during the session.

5.3 Experiment Design

We employed a within-subjects design with one independent variable: keyboard type. The study compared two keyboard types: the conventional Braille keyboard and our intelligent Braille keyboard. The conventional Braille keyboard used in the study was the built-in TalkBack Braille keyboard available on the Google Pixel 6 with Android 14. We adopted a text transcription task, where participants transcribe a set of phrases using our web-based keyboard. The task has a 10 minutes warm-up session starting from writing individual characters to typing two complete phrases, then the participants were given a formal test session with 20 phrases.

The warm-up phrases were randomly picked from a versatile phrase set for evaluating typing task [58]. The test phrases were randomly picked from the T-40 dataset [60]. Participants are required to type the 26 lowercase letters, with no need to type capital, number, special symbols or punctuation, but a space between each word is demanded. All phrases were the same for each participant but presented in a random order. To control for order effects, participants were evenly divided into two conditions: starting with the conventional Braille keyboard or our intelligent Braille keyboard.

We developed a webpage to shuffle and read out the target phrases. Figure 6 shows the interface used in the transcription task for the formal session in the user study. The experimenter refreshed this page for each participant to present a shuffled set of phrases and click the "Read a Random Phrase" button to select a new phrase, while the "Read Again" button repeats the current phrase as needed by the participant. The interface tracks the number of sentences read out of 20. We used the same interface for the warm-up session with a different set of phrases [58].

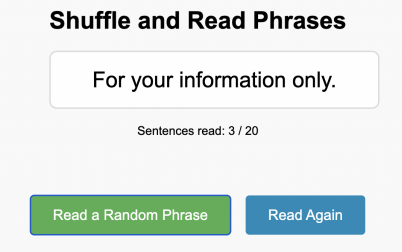


Figure 6: A webpage screenshot of our transcription task

5.4 Procedure

After greeting the participants, we introduced the purpose and obtain informed consent. We conducted a pre-study interview to collect demographic information. At the beginning of the study, we provided instructions on how to hold the smartphone, how to calibrate the Braille dots layout, and how to input Braille and swipe to type a space or loop over the suggestion list. Then we held a warm-up session to tap the Braille alphabet followed by two phrases, it takes about 10 minutes. During the formal study, participants were instructed to complete the transcription task as accurately and quickly as possible in a comfortable position. After completing a phrase, the experimenter will play the next phrase on the developed webpage. Participants could take a break after every five phrases if needed. Following each keyboard task, a post-study interview assessed participants' perceptions of usability using the SUS questionnaire [17]. This well-established questionnaire is a valuable tool for measuring user satisfaction with a system.

At the end of the study, participants also engaged in an informal discussion aimed at gathering open-ended feedback on their experience and suggestions for improving soft Braille keyboards. The user study lasted approximately two hours.

5.5 Results

All 12 participants successfully completed typing 20 phrases on the conventional Braille keyboard and our intelligent Braille keyboard.

5.5.1 ***Input Speed.*** We first investigated the input speed measured in WPM [38]. The average WPM across all participants for the conventional Braille keyboard was 6.18 (SD = 1.80) and 5.44 (SD = 1.73) for the intelligent Braille keyboard. The conventional Braille keyboard has a faster average typing speed than the intelligent Braille keyboard. This difference is expected, as the intelligent Braille keyboard requires additional time to select the suggested word when errors occur. There is an observation that some participants were not accustomed to using the swipe-up or swipe-down gestures to cycle through the suggestion list. This unfamiliarity often resulted in additional time spent adjusting their hand posture and calibrating the Braille dots on the screen after swiping. Consequently, this process disrupted their typing flow, leading to slower typing speeds.

A paired t-test showed a significant main effect for the keyboard type ($t_{11} = 3.40, p < 0.01$) with a small effect size (Cohen's d = 0.42) on the WPM. The average typing speed of a conventional Braille keyboard was 6.18 WPM, 0.74 WPM higher than the intelligent Braille keyboard (5.44 WPM). However, a few participants (P4, P9) achieve similar or slightly better speeds on the intelligent keyboard, indicating potential benefits for some users who might leverage its features more effectively.

5.5.2 ***Error Rate.*** We investigated the WER [10] using Minimum Word Distance (MWD) for each keyboard condition. The average WER across all participants for the conventional Braille keyboard was 42.53% (SD=13.51%) and 17.28% (SD=12.76%) for the intelligent Braille keyboard. As shown in Figure 7, the intelligent Braille keyboard resulted in lower WER compared to the conventional Braille keyboard. This suggests that our algorithm contributed to

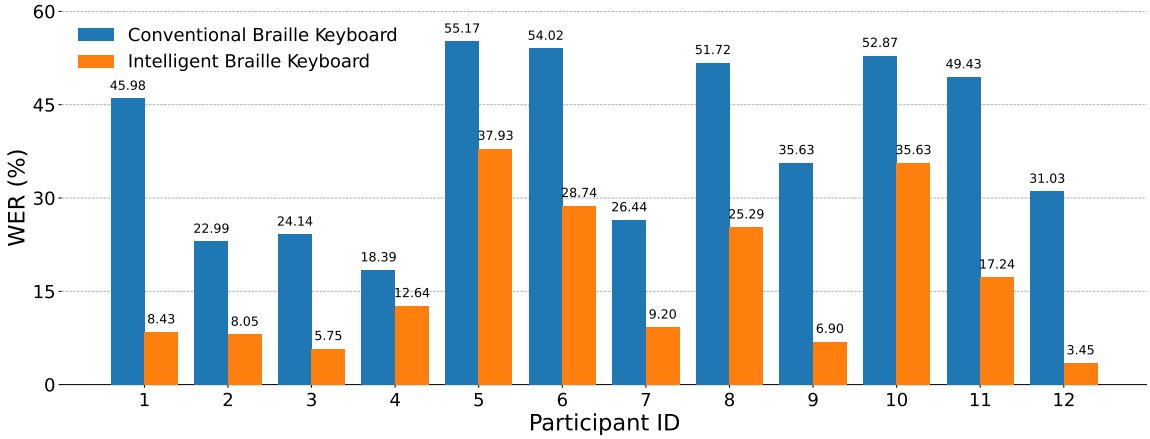


Figure 7: Average WER for the 12 participants

Table 4: Analysis of Suggestion Usage: Frequency of Swipe-Up Gestures and Correct Suggestions Adopted

	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	Mean
Suggestion Usage	17	14	15	24	36	25	16	30	13	45	25	19	**23.3**
Correct Suggestion	15	10	12	17	25	15	15	24	9	29	17	15	**16.9**
Accuracy Rate (%)	88.2	71.4	80.0	70.8	69.4	60.00	93.8	80.0	69.2	64.4	68.0	78.9	**74.5**

fewer errors. It is important to note that backspace correction was intentionally disabled for both conditions to enable controlled measurement of raw input accuracy. As such, the error rates observed here may be higher than those in real-world settings, where users are free to manually correct mistakes. This design choice isolates the auto-correction capabilities of the keyboard, and the reduction in WER highlights the benefit of integrating our intelligent decoding approach.

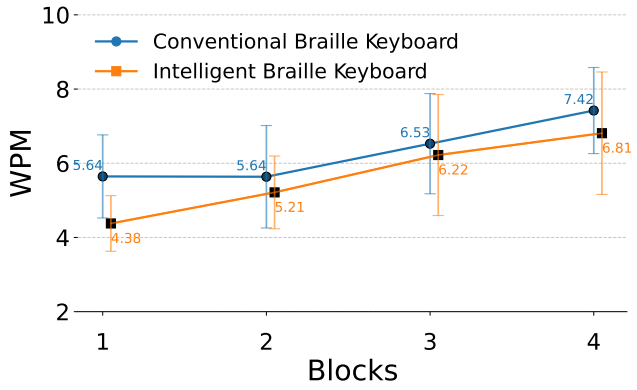
To statistically analyze these observations, we conducted a paired t-test. The results showed significant main effects for the keyboard type ($t_{11} = 9.56, p < 0.001$) with a very large effect size (Cohen's d = 1.90) on the WER. These results suggest that the intelligent keyboard significantly impacted the typing error rate. This highlights the importance of integrating correction features with the Braille keyboard to reduce typing errors. Figure 7 shows the WPM of the 12 participants using both the conventional Braille keyboard and the intelligent Braille keyboard. It is evident that there is a noticeable reduction in WER for most participants when using the intelligent keyboard compared to the conventional keyboard. However, the extent of this improvement varies across participants, indicating that while the intelligent keyboard generally improves typing accuracy, the level of benefit may depend on individual user factors such as familiarity with the correction features, typing proficiency, and adaptation to the intelligent keyboard's interface.

5.5.3 ***Suggestion Usage***. We recorded the frequency of swipe-up gestures performed by participants to access the suggestion list while typing. This metric provides insight into how frequently participants relied on the suggestion feature during text entry. To better understand the effectiveness of these suggestions, we compared the number of times participants accessed the suggestion list with the number of correct words that were selected from the list.

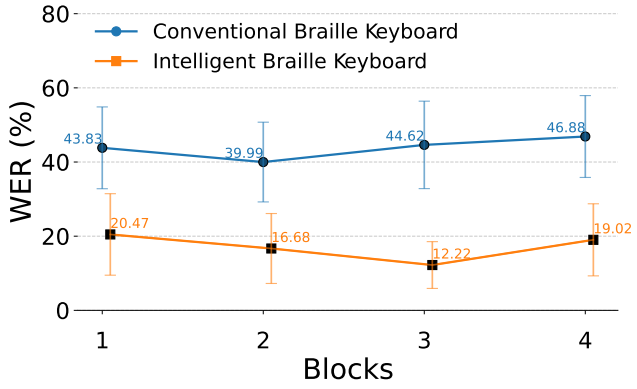
The results of this analysis are presented in Table 4. The table shows the total number of swipe-up gestures, the number of correct suggestions adopted, and the percentage of correct suggestions relative to the total gestures for each participant. The frequency of usage varies, with some participants utilizing the suggestion feature more frequently than others. The accuracy rates indicate that the suggestion function is indeed effective in improving typing accuracy, with the percentage of correct suggestions ranging from 60.0% to 93.8% across participants. This suggests that while there is variability in reliance on the suggestion feature, it generally contributes positively to typing accuracy.

5.5.4 ***Learning Effect***. To evaluate participants' performance over time, we analyzed their typing speed and error rate across four blocks, each comprising five phrases. We computed the WPM and WER with 95% confidence intervals (CI) for each block on the conventional Braille keyboard and the intelligent Braille Keyboard. As illustrated in Figure 8, the graph of the mean input speed (95% CI) across the four blocks provides a clear depiction of how the typing speeds and error rates of the participants evolved.

A repeated measures ANOVA showed significant learning effects for the blocks on the WPM ($F_{3,\ 33} = 15.27, p < 0.001$ for the standard keyboard, $F_{3,\ 33} = 11.85, p < 0.001$ for the intelligent keyboard). Post-hoc pairwise comparison with Holm adjustment showed that the text entry speeds in session 3 and 4 were significantly higher than session 1 and session 2, which indicates that users could achieve a relatively high speed after practicing for only 5-10 sentences.



(a) WPM



(b) WER

Figure 8: Average(95%CI) of WPM and WER(%) by block

A repeated measures ANOVA revealed no statistically significant main effect of blocks on WER (($F_{3,\ 33} = 0.765, p = 0.522$) for the standard keyboard; ($F_{3,\ 33} = 1.844, p = 0.158$) for the intelligent keyboard). The intelligent Braille keyboard initially resulted in a higher WER but showed a marked improvement by Block 3. The ability to maintain a relatively low WER while increasing typing speed suggests that the intelligent keyboard may provide effective error correction mechanisms that users gradually learn to utilize. However, the slight increase in both WPM and WER in Block 4 suggests that participants may have experienced fatigue or that further improvements to the algorithm might be needed to maintain high accuracy levels.

5.5.5 ***Usability***. We report here the SUS questionnaire results [17]. The median SUS score for the conventional Braille keyboard is 71.25, and 81.5 for the intelligent Braille Keyboard. A Friedman test revealed a statistically significant main effect of keyboard types on SUS scores ($\chi^2(1) = 12, p < 0.001$). Given that multitouch Braille typing can be both fast and error prone [52], the improved SUS scores for the intelligent keyboard suggest that advanced correction capabilities are essential for reducing errors and increasing overall typing efficiency. In support of this, some participants provided positive feedback (lightly edited for grammar without changing their meaning): "I like the suggestions; typing correctly is important to me" (P2), "Swiping up and down to search for the correct words is easy and natural" (P3), and "The integration of the suggestion features is a good implementation" (P11).

6 Discussion

6.1 Adopting the Intelligent Braille Decoder for the Braille Keyboard

The major finding from our simulation-based and user studies is that the intelligent Braille decoder drastically reduced the input errors over the existing Braille keyboards. For people with visual impairment, one critical requirement for a Braille decoder is to have high accuracy for the Top-1 suggestion. As it is slow for a visually impaired individual to navigate through the suggestion lists, high accuracy in Top-1 suggestion would save navigation time. Therefore, the large improvement of Top-1 suggestion accuracy of the proposed intelligent Braille decoder (Table 1, Table 2) brings big benefits to users.

Although onscreen Braille keyboards have been available on smartphones for years [28, 34], they often lack the intelligence to correct typing errors. Our finding highlights the value of corrective capabilities in enhancing typing accuracy for visually impaired users. While the intelligent keyboard introduced a slight decrease in typing speed, this trade-off is justified by the significant improvement in accuracy, especially for users who prioritize correctness over speed. Given its promising performance, we expect the proposed Braille decoder for a Braille soft keyboard would work as the statistical decoder for a soft QWERTY keyboard. The statistical decoder has been widely adopted by almost every commercial soft QWERTY keyboard, and is by default turned on for every user. We hope the decoding technique proposed by this work will also adopted by existing Braille soft keyboards, benefiting the large number of people with visual impairments.

6.2 Differences Across Individuals

Our user study reveals substantial variations among participants in terms of WER on both conventional and intelligent Braille keyboards (Figure 7) and the frequency of suggestion usage (Table 4). This is likely attributed to the absence of visual feedback, causing users to depend on their perception of Braille dot placements for touch point accuracy, leading to diverse typing behaviors. This insight emphasizes the necessity of considering individual differences to enhance typing experiences. For example, users like P8, who frequently relied on word suggestions, might benefit from more efficient mechanisms such as gesture shortcuts or voice feedback for quickly cycling through suggestions. Personalization could further enhance the experience by allowing users to configure the gesture controls to fit their preferences. Collecting individual typing behavior data could also enable the decoder to better align with a user's unique input patterns. Overall, supporting user-specific adaptation is essential for making Braille typing more accurate, comfortable, and efficient.

6.3 Promoting Braille Usage via the Intelligent Braille Keyboard

We hope the improved typing performance on intelligent Braille keyboard would encourage more and more people to use Braille on smartphones, increasing the Braille literacy among people with visual impairments. The Braille writing system is crucial for people with visual impairments to read, write, and access information. Although it is essential, merely a small fraction of people with visual impairments (approximately 10%) are proficient in Braille [44]. The National Federation of the Blind has identified the decline in Braille literacy as a crisis and has been calling for swift actions to reverse it [44].

The intelligent Braille keyboard has the potential to positively influence Braille usage, particularly on touchscreen devices such as smartphones and tablets. Since text entry is a central activity in daily smartphone use, ranging from messaging and emailing to social media, tools that improve typing efficiency and accuracy may help make Braille input more appealing and accessible. While it is premature to claim that such technology alone will reverse the decline in Braille literacy, reducing barriers to effective Braille typing could support broader efforts to encourage Braille use, especially among individuals who already have some familiarity with the system. In this way, improved typing experiences may serve as one component in fostering more regular engagement with Braille in digital contexts.

6.4 Comparison to Other Input Methods

The intelligent Braille keyboard achieved an average input speed of 5.44 WPM, reaching 6.81 WPM in the final block, with an average WER of 17.28%. This performance is comparable to existing methods such as Qwerty with search-and-confirm (4–5 WPM, 25% WER) [43] and AGTex with gesture typing (5.66 WPM, 14.87% WER) [13]. These results indicate that our method offers a viable and competitive alternative for blind users. By narrowing the performance gap between Braille and other input techniques, the intelligent Braille keyboard may help legitimize Braille typing as a practical everyday option on smartphones. This, in turn, could contribute to sustaining Braille usage in the digital age, where efficient and accessible text input remains a foundational need for blind users.

6.5 Limitations and Future Work

While the proposed intelligent Braille keyboard demonstrates promising performance, there are several limitations to consider. First, users experienced slower typing speeds compared to conventional Braille keyboards, primarily due to the time spent selecting suggested words. To address this, future research could focus on optimizing the suggestion interface to minimize interaction time. Additionally, incorporating advanced predictive typing techniques, such as personalized suggestions, could further enhance typing efficiency and user experience.

Second, the intelligent keyboard encountered difficulties with positional errors ("Disposition Dot" in Table 2 and Table 3), highlighting a potential area for improvement. Addressing this challenge requires advancements in the decoder's capabilities. Integrating larger, more robust language models or neural networks could significantly enhance the keyboard's ability to interpret positional errors and infer the intended input. Additionally, tactile feedback such as Braille dot stickers for Braille dots could further improve accuracy. By providing a physical sensation, users can better orient their fingers on the screen and confirm that their intended input has been entered.

The next crucial step involves implementing the intelligent keyboard as a fully functional application and deploying it on mobile devices for real-world testing. Direct feedback from users in real-world contexts is essential for refining the interface and features. Conducting larger-scale studies with more diverse participant groups, including users with varying levels of Braille proficiency and different demographic backgrounds, will be necessary to validate the keyboard's effectiveness.

Another interesting direction is to investigate how to leverage LLMs such as ChatGPT [46] or Gemini [23] to further improve the typing experience a Braille soft keyboard. For example, it is worth experimenting using an LLM such as ChatGPT to provides the language model score, rather than a bigram language model in the decoding process. Or we may use an LLM to further correct the text after receiving the decoding results from the proposed intelligent Braille decoder. Both are promising future research directions to improve the performance of Braille decoding.

7 Conclusion

We designed and implemented a novel intelligent Braille keyboard with auto-correction for smartphones that can automatically correct errors while typing in Braille. We created an intelligent decoder that uses optimal transportation theory to calculate the distances between touch input and Braille patterns, and combine it with a language model to estimate the probability of entering a word. We compared the error correction capability of our intelligent Braille keyboard powered by the proposed decoder with that of the existing TalkBack Braille keyboard and the iPhone Braille keyboard by simulating touch interactions on an Android smartphone and an iPhone. The results showed that our intelligent Braille keyboard greatly reduces typing errors compared to the Android's proofread feature and the iPhone's spellchecker for Braille input. It reduced the error rate from 55.81% (Top-1) and 52.74% (Top-3) on Android and 57.13% (Top-1) and 52.82% (Top-3) on iPhone to just 19.80% (Top-1) and 12.29% (Top-3) under high typing noise conditions (SD = 7.8 mm), and demonstrated very low error rates under lower typing noise (SD = 3.9 mm). Furthermore, in a user study involving 12 participants, we compared the performance of our keyboard against a conventional Braille keyboard without auto-correction. The results indicated that our keyboard significantly reduced the word error rate, from 42.53% to 17.28%. The Braille keyboard provides a promising method for accurate text entry for blind users on smartphones.

Acknowledgments

We thank the anonymous reviewers for their constructive feedback and the user study participants for generously sharing their time and insights. This work was supported by NSF awards 2113485, 2153056, NIH awards R01EY03568801, R01EY03008503, and DoD-USAMRAA award HT94252410098.

References

[1] Mrim Alnfiai and Srinivas Sampalli. 2016. SingleTapBraille: Developing a text entry method based on braille patterns using a single tap. *Procedia Computer Science* 94 (2016), 248–255.

[2] Mrim Alnfiai and Srinivas Sampalli. 2017. BrailleEnter: a touch screen braille text entry method for the blind. *Procedia Computer Science* 109 (2017), 257–264.

[3] Jason Altschuler, Jonathan Niles-Weed, and Philippe Rigollet. 2017. Near-linear time approximation algorithms for optimal transport via Sinkhorn iteration. *Advances in neural information processing systems* 30 (2017).

[4] Martin Arjovsky, Soumith Chintala, and Léon Bottou. 2017. Wasserstein generative adversarial networks. In *International conference on machine learning*. PMLR, 214–223.

[5] AutoTouch. 2024. *Release Notes: Version 8.5.1.* https://docs.autotouch.net/release-notes/#v8-5-1 Accessed: 2024-12-08.

[6] AutoTouch LLC. [n. d.]. *AutoTouch Homepage.* https://autotouch.net/ Accessed: 2024-09-08.

[7] Shiri Azenkot. 2014. *Eyes-Free input on mobile devices.* Ph. D. Dissertation.

[8] Shiri Azenkot and Nicole B. Lee. 2013. Exploring the Use of Speech Input by Blind People on Mobile Devices. In *Proceedings of the 15th International ACM SIGACCESS Conference on Computers and Accessibility* (Bellevue, Washington) *(ASSETS '13)*. Association for Computing Machinery, New York, NY, USA, Article 11, 8 pages. doi:10.1145/2513383.2513440

[9] Shiri Azenkot, Jacob O. Wobbrock, Sanjana Prasain, and Richard E. Ladner. 2012. Input Finger Detection for Nonvisual Touch Screen Text Entry in Perkinput. In *Proceedings of Graphics Interface 2012* (Toronto, Ontario, Canada) *(GI '12)*. Canadian Information Processing Society, Toronto, Ont., Canada, Canada, 121–129. http://dl.acm.org/citation.cfm?id=2305276.2305297

[10] Xiaojun Bi, Shiri Azenkot, Kurt Partridge, and Shumin Zhai. 2013. Octopus: evaluating touchscreen keyboard correction and recognition algorithms via "Remulation". In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 543–552.

[11] Xiaojun Bi, Yang Li, and Shumin Zhai. 2013. FFitts Law: Modeling Finger Touch with Fitts' Law. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Paris, France) *(CHI '13)*. ACM, New York, NY, USA, 1363–1372. doi:10.1145/2470654.2466180

[12] Xiaojun Bi and Shumin Zhai. 2016. Predicting Finger-Touch Accuracy Based on the Dual Gaussian Distribution Model. In *Proceedings of the 29th Annual Symposium on User Interface Software and Technology* (Tokyo, Japan) *(UIST '16)*. ACM, New York, NY, USA, 313–319. doi:10.1145/2984511.2984546

[13] Syed Masum Billah, Yu-Jung Ko, Vikas Ashok, Xiaojun Bi, and IV Ramakrishnan. 2019. Accessible gesture typing for non-visual text entry on smartphones. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. 1–12.

[14] Nicolas Bonneel, Michiel Van De Panne, Sylvain Paris, and Wolfgang Heidrich. 2011. Displacement interpolation using Lagrangian mass transport. In *Proceedings of the 2011 SIGGRAPH Asia conference*. 1–12.

[15] Matthew N. Bonner, Jeremy T. Brudvik, Gregory D. Abowd, and W. Keith Edwards. 2010. No-look Notes: Accessible Eyes-free Multi-touch Text Entry. In *Proceedings of the 8th International Conference on Pervasive Computing* (Helsinki, Finland) *(Pervasive'10)*. Springer-Verlag, Berlin, Heidelberg, 409–426. doi:10.1007/978-3-642-12654-3_24

[16] Louis Braille. 1829. Method of writing words, music, and plain songs by means of dots, for use by the blind and arranged for them. *Paris, France: l'Institution Royale des Jeunes Aveugles* (1829).

[17] John Brooke. 1996. Sus: a "quick and dirty'usability. *Usability evaluation in industry* 189, 3 (1996).

[18] Barry Brown, Moira McGregor, and Donald McMillan. 2014. 100 days of iPhone use: understanding the details of mobile device use. In *Proceedings of the 16th international conference on Human-computer interaction with mobile devices & services*. 223–232.

[19] Appium Contributors. 2024. *Appium: An Open-Source Mobile Testing Framework.* https://appium.io Accessed: 2024-11-27.

[20] Nicolas Courty, Rémi Flamary, Devis Tuia, and Alain Rakotomamonjy. 2016. Optimal transport for domain adaptation. *IEEE transactions on pattern analysis and machine intelligence* 39, 9 (2016), 1853–1865.

[21] Marco Cuturi. 2013. Sinkhorn distances: Lightspeed computation of optimal transport. *Advances in neural information processing systems* 26 (2013).

[22] Mark Davies. 2008. The corpus of contemporary American English: 1990-present.

[23] Google DeepMind. 2024. *Gemini: The Next-Generation AI by DeepMind.* https://www.deepmind.com Accessed: 2024-12-08.

[24] Rémi Flamary, Nicolas Courty, Alexandre Gramfort, Mokhtar Z. Alaya, Aurélie Boisbunon, Stanislas Chambon, Laetitia Chapel, Adrien Corenflos, Kilian Fatras, Nemo Fournier, Léo Gautheron, Nathalie T.H. Gayraud, Hicham Janati, Alain Rakotomamonjy, Ievgen Redko, Antoine Rolet, Antony Schutz, Vivien Seguy, Danica J. Sutherland, Romain Tavenard, Alexander Tong, and Titouan Vayer. 2021. POT: Python Optimal Transport. *Journal of Machine Learning Research* 22, 78 (2021), 1–8. http://jmlr.org/papers/v22/20-451.html

[25] Andrew Fowler, Kurt Partridge, Ciprian Chelba, Xiaojun Bi, Tom Ouyang, and Shumin Zhai. 2015. Effects of Language Modeling and Its Personalization on Touchscreen Typing Performance. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems* (Seoul, Republic of Korea) *(CHI '15)*. ACM, New York, NY, USA, 649–658. doi:10.1145/2702123.2702503

[26] Brian Frey, Caleb Southern, and Mario Romero. 2011. Brailletouch: Mobile Texting for the Visually Impaired. In *Proceedings of the 6th International Conference on Universal Access in Human-computer Interaction: Context Diversity - Volume Part III* (Orlando, FL) *(UAHCI'11)*. Springer-Verlag, Berlin, Heidelberg, 19–25. http://dl.acm.org/citation.cfm?id=2022539.2022543

[27] Joshua Goodman, Gina Venolia, Keith Steury, and Chauncey Parker. 2002. Language Modeling for Soft Keyboards. In *Proceedings of the 7th International Conference on Intelligent User Interfaces* (San Francisco, California, USA) *(IUI '02)*. ACM, New York, NY, USA, 194–195. doi:10.1145/502716.502753

[28] Google LLC. [n. d.]. *About Android 5.0 Updates.* https://www.android.com/versions/lollipop-5-0/ Accessed: 2024-09-08.

[29] Asela Gunawardana, Tim Paek, and Christopher Meek. 2010. Usability Guided Key-target Resizing for Soft Keyboards. In *Proceedings of the 15th International Conference on Intelligent User Interfaces* (Hong Kong, China) *(IUI '10)*. ACM, New York, NY, USA, 111–118. doi:10.1145/1719970.1719986

[30] Google Accessibility Help. 2024. *Braille keyboard instructions on Android.* https://support.google.com/accessibility/android/answer/14252962?hl=en

[31] Christian Holz and Patrick Baudisch. 2011. Understanding Touch. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Vancouver, BC, Canada) *(CHI '11)*. ACM, New York, NY, USA, 2501–2510. doi:10.1145/1978942.1979308

[32] Jonggi Hong, Christine Vaing, Hernisa Kacorri, and Leah Findlater. 2020. Reviewing Speech Input with Audio: Differences between Blind and Sighted Users. *ACM Transactions on Accessible Computing (TACCESS)* 13, 1 (2020), 1–28.

[33] Le Hou, Chen-Ping Yu, and Dimitris Samaras. 2016. Squared earth mover's distance-based loss for training deep neural networks. *arXiv preprint arXiv:1611.05916* (2016).

[34] Apple Inc. 2016. Apple Accessibility: VoiceOver Getting Started Guide. *Product Manual* (2016). Online; accessed 16-July-2024. https://help.apple.com/voiceover/info/guide/10.12/.

[35] Brian Kemler. [n. d.]. A new keyboard for typing braille on Android. https://blog.google/products/android/braille-keyboard/ Published in April 09, 2020; Retrieved in Feb 2023.

[36] Per-Ola Kristensson and Shumin Zhai. 2004. SHARK2: a large vocabulary shorthand writing system for pen-based computers. In *Proceedings of the 17th annual ACM symposium on User interface software and technology*. 43–52.

[37] Yan Ma, Shumin Zhai, IV Ramakrishnan, and Xiaojun Bi. 2021. Modeling touch point distribution with rotational dual gaussian model. In *The 34th annual acm symposium on user interface software and technology*. 1197–1209.

[38] I. Scott MacKenzie. 2015. *A Note on Calculating Text Entry Speed.*

[39] Sergio Mascetti, Cristian Bernareggi, and Matteo Belotti. 2011. TypeInBraille: Quick Typing on Smart Phones by Blind Users. *Recuperado de http://homes. di. unimi. it/˜ mascetti/Sergio_Mascetti_-_home_page/Research_files/TR39-2011. pdf* (2011).

[40] Elke Mattheiss, Georg Regal, Johann Schrammel, Markus Garschall, and Manfred Tscheligi. 2014. Dots and letters: Accessible braille-based text input for visually impaired people on mobile touchscreen devices. In *Computers Helping People with Special Needs: 14th International Conference, ICCHP 2014, Paris, France, July 9-11, 2014, Proceedings, Part I 14*. Springer, 650–657.

[41] Elke Mattheiss, Georg Regal, Johann Schrammel, Markus Garschall, and Manfred Tscheligi. 2015. EdgeBraille: Braille-based text input for touch devices. *Journal of Assistive Technologies* 9, 3 (2015), 147–158.

[42] Hugo Nicolau, Kyle Montague, Tiago Guerreiro, João Guerreiro, and Vicki L Hanson. 2014. B# chord-based correction for multitouch braille input. In *Proceedings of the SIGCHI conference on human factors in computing systems*. 1705–1708.

[43] Hugo Nicolau, Kyle Montague, Tiago Guerreiro, André Rodrigues, and Vicki L Hanson. 2015. Typing performance of blind users: An analysis of touch behaviors, learning effect, and in-situ usage. In *Proceedings of the 17th International ACM SIGACCESS Conference on Computers & Accessibility*. 273–280.

[44] National Federation of the Blind. 2009. The braille literacy crisis in America: Facing the truth, reversing the trend, empowering the blind. *Braille Monitor* 52, 5 (2009).

[45] João Oliveira, Tiago Guerreiro, Hugo Nicolau, Joaquim Jorge, and Daniel Gonçalves. 2011. BrailleType: Unleashing Braille over Touch Screen Mobile Phones. In *Proceedings of the 13th IFIP TC 13 International Conference on Human-computer Interaction - Volume Part I* (Lisbon, Portugal) *(INTERACT'11)*. Springer, Springer-Verlag, Berlin, Heidelberg, 100–107. http://dl.acm.org/citation.cfm?id=2042053.2042066

[46] OpenAI. 2024. *ChatGPT: Language Model for Dialogue Applications.* https://openai.com/chatgpt Accessed: 2024-12-08.

[47] Kseniia Palin, Anna Feit, Sunjun Kim, Per Ola Kristensson, and Antti Oulasvirta. 2019. How do People Type on Mobile Devices? Observations from a Study with

37,000 Volunteers.. In *Proceedings of 21st International Conference on Human-Computer Interaction with Mobile Devices and Services (MobileHCI'19)*. ACM. doi:10.475/123_4

[48] Pew Research Center. [n. d.]. U.S. Smartphone Use in 2015. http: //www.pewinternet.org/2015/04/01/us-smartphone-use-in-2015/ Accessed in Feb 2023.

[49] Gabriel Peyré, Marco Cuturi, et al. 2019. Computational optimal transport: With applications to data science. *Foundations and Trends® in Machine Learning* 11, 5-6 (2019), 355–607.

[50] Filippo Santambrogio. 2015. Optimal transport for applied mathematicians. *Birkäuser, NY* 55, 58-63 (2015), 94.

[51] Shai Shalev-Shwartz and Ambuj Tewari. 2009. Stochastic methods for l 1 regularized loss minimization. In *Proceedings of the 26th Annual International Conference on Machine Learning*. 929–936.

[52] Caleb Southern, James Clawson, Brian Frey, Gregory Abowd, and Mario Romero. 2012. An evaluation of BrailleTouch: mobile touchscreen text entry for the visually impaired. In *Proceedings of the 14th international conference on Human-computer interaction with mobile devices and services*. 317–326.

[53] Nikhil Siva Subash, Siddhartha Nambiar, and Vishesh Kumar. 2012. Braillekey: An alternative braille text input system: Comparative study of an innovative simplified text input system for the visually impaired. In *2012 4th International Conference on Intelligent Human Computer Interaction (IHCI)*. IEEE, 1–4.

[54] Apple Support. n.d.. *Accessibility features for Apple devices.* https://support.apple.com/en-us/101637

[55] TalkBack. [n. d.]. TalkBack: An Open Source Screenreader For Android. http://google-opensource.blogspot.com/2009/10/talkback-open-source-screenreader-for.html Online. http://google-opensource.blogspot.com/2009/10/talkback-open-source-screenreader-for.html.

[56] Python Optimal Transport. 2024. *Quickstart - Optimal Transport and Wasserstein Distance.* https://pythonot.github.io/quickstart.html#optimal-transport-and-wasserstein-distance Accessed: 2024-12-08.

[57] Keith Vertanen, Crystal Fletcher, Dylan Gaines, Jacob Gould, and Per Ola Kristensson. 2018. The Impact of Word, Multiple Word, and Sentence Input on Virtual Keyboard Decoding Performance. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems* (Montreal QC, Canada) *(CHI '18)*. ACM, New York, NY, USA, Article 626, 12 pages. doi:10.1145/3173574.3174200

[58] Keith Vertanen and Per Ola Kristensson. 2011. A versatile dataset for text entry evaluations based on genuine mobile emails. In *Proceedings of the 13th International Conference on Human Computer Interaction with Mobile Devices and Services*. 295–298.

[59] Cédric Villani et al. 2009. *Optimal transport: old and new*. Vol. 338. Springer.

[60] Xin Yi, Chun Yu, Weinan Shi, Xiaojun Bi, and Yuanchun Shi. 2017. Word clarity as a metric in sampling keyboard test sets. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. 4216–4228.

[61] Shumin Zhai and Per Ola Kristensson. 2012. The Word-gesture Keyboard: Reimagining Keyboard Interaction. *Commun. ACM* 55, 9 (Sept. 2012), 91–101. doi:10.1145/2330667.2330689

[62] Dan Zhang, Zhi Li, Vikas Ashok, William H Seiple, IV Ramakrishnan, and Xiaojun Bi. 2024. Accessible gesture typing on smartphones for people with low vision. In *Proceedings of the 37th annual ACM symposium on User interface software and technology*.

[63] Suwen Zhu, Tianyao Luo, Xiaojun Bi, and Shumin Zhai. 2018. Typing on an invisible keyboard. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. 1–13.